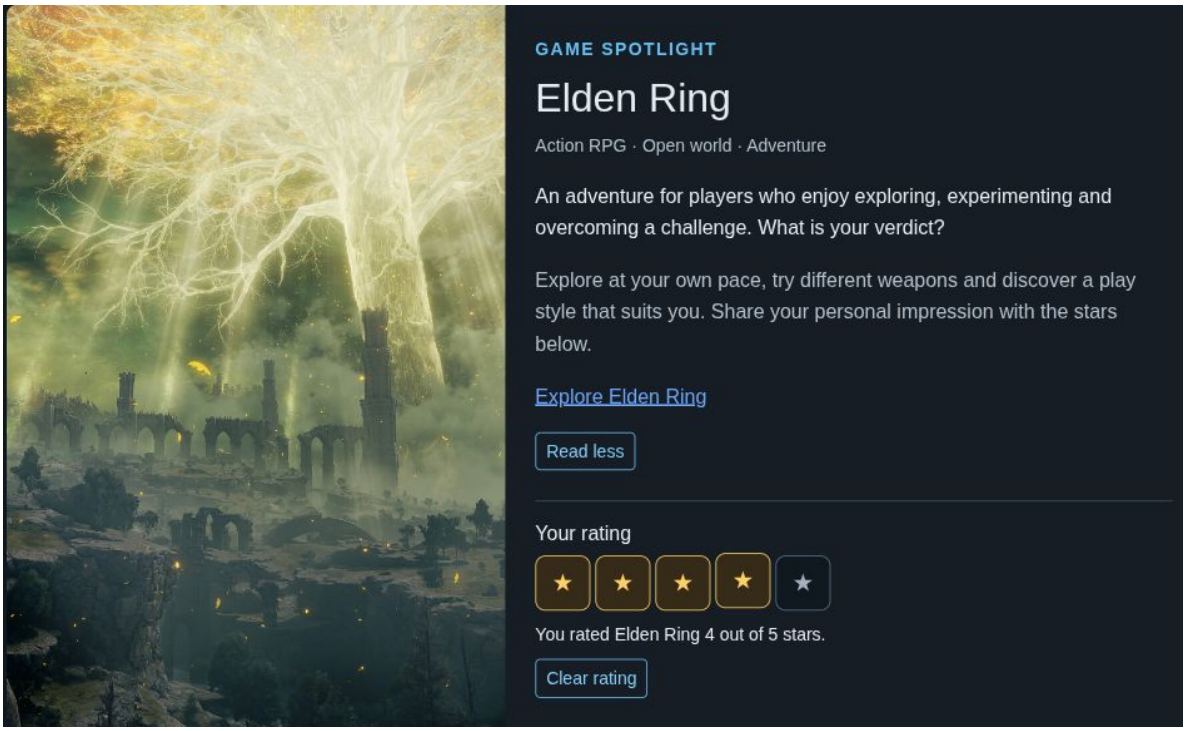
ASSIGNMENT 5 / PROJECT REPORT

Advanced JavaScript & DOM

Steam Game Hub extends the existing team website with interactive game discovery and a local player-card builder. The project continues Assignments 3 and 4 using plain HTML, CSS and JavaScript.

Team: Eggs **Group:** SE-2507
Course: Web Technologies I
Members: Ташкенбаев Нурсултан; Бигали Абулхаир;
Султанкулов Шынгыс; Лопаткин Артем.

Objective. Demonstrate DOM selection and modification, event listeners, keyboard navigation, callbacks, switch statements, objects, arrays, loops, higher-order functions, audio and event-triggered animation in a coherent gaming interface.



Home: the player selects four stars and expands the game description.

Website: https://abilkhayrbigali-design.github.io/
Repository: github.com/abilkhayrbigali-design/abilkhayrbigali-design.github.io

Submission: this PDF and Assignment_5_Project.zip. Extract the ZIP and open index.html. The site includes local Bootstrap, game images and its notification sound; no build step is required.

Requirement coverage

All mandatory areas are covered. Where the brief offers alternatives, the project uses Read more for attribute/visibility manipulation, direct style changes, a multi-step form for callbacks, and a catalogue price filter for switch logic.

Requirement	Implementation / demonstration
DOM selection and content	Home: querySelectorAll, querySelector, getElementById and textContent in the rating and description.
Dynamic styling	Home: selected-star classes and the existing background palette.
Attribute / visibility option	Read more updates style.display and aria-expanded.
Button events	Clear rating, Reset filters, Next, Back and Play notification use addEventListener.
Keyboard events	Arrow keys wrap focus within navigation; Home/End jump to the endpoints.
Callback function	Community: state.goTo(step, showStep) calls the supplied function and retains input values.
Switch statement	Games: matchesBudget chooses Free, paid under $30, $30+ or all prices.
Objects and methods	catalogue.games plus catalogue.findGames(); state.step plus state.goTo().
Arrays and loops	map creates game objects; for...of updates card visibility.
Higher-order function	filter receives a predicate callback; map and forEach also receive callbacks.
Sound and animation	Local 1.2-second chime; animated sound bars and filtered-card entrance effects.
Structure and UX	Separate HTML/CSS/JS, named functions, clear labels, focus indicators, responsive layouts.
Deployment and report	GitHub Pages from main; screenshots, test evidence, reflection and submission archive.

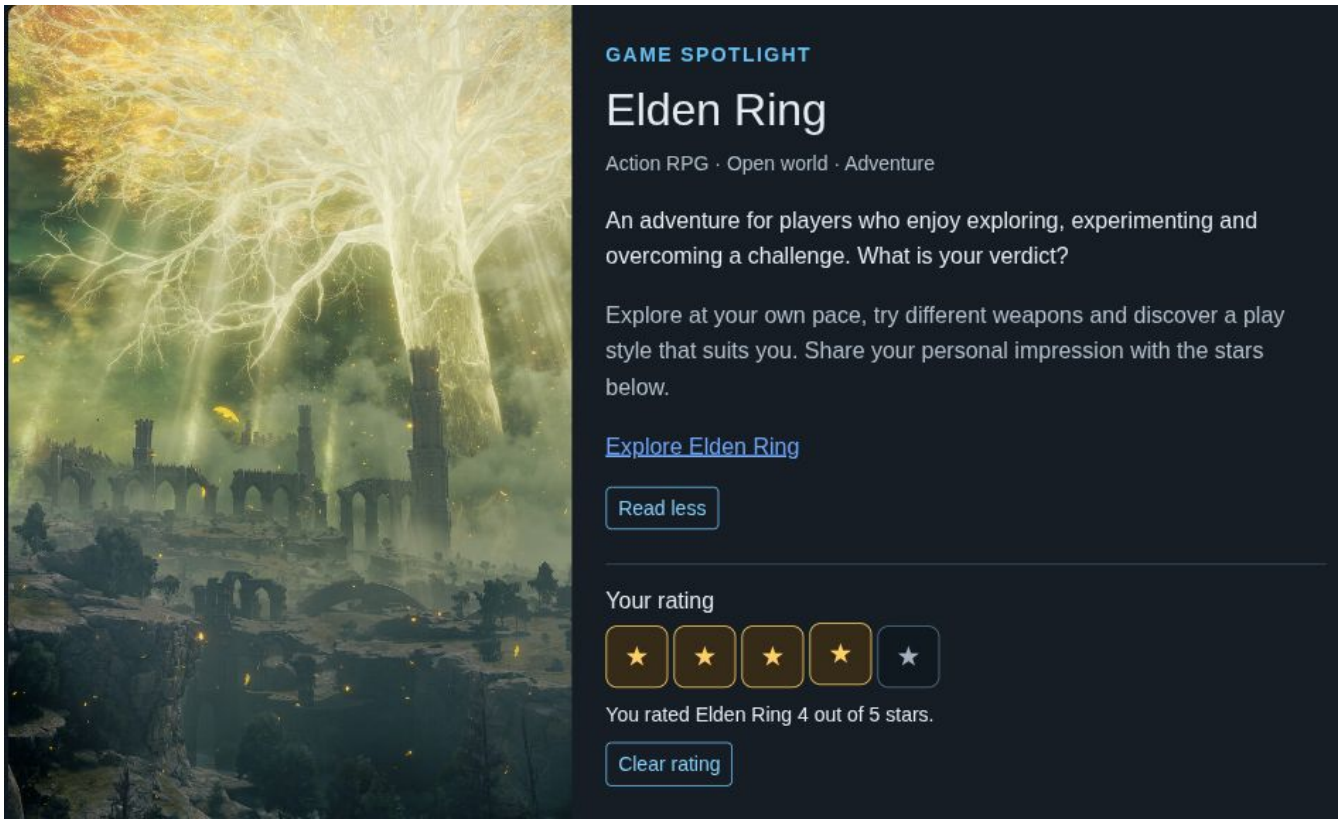
1. Select and update the DOM

Steps. (1) Add a labelled group of five radio inputs. (2) Select its inputs, star labels and status element. (3) On change, convert the selected value to a number. (4) Toggle the selected class on every star up to that number and update the message with textContent.

js/assignment5.js

```
4     document.querySelectorAll('[data-game-rating]').forEach((rating) ⇒ {
5       const inputs = rating.querySelectorAll('input[type="radio"]');
6       const stars = rating.querySelectorAll('[data-rating-star]');
7       const status = rating.querySelector('[role="status"]');
8       const resetButton = rating.querySelector('[data-reset-rating]');
9
10      function updateRating(value) {
11        stars.forEach((star) ⇒ {
12          star.classList.toggle('is-selected', Number(star.dataset.ratingStar) ⩽ value);
13        });
14        status.textContent = value > 0
15          ? `You rated ${rating.dataset.gameRating} ${value} out of 5 stars.`
16          : 'Choose a star to rate this game.';
17        resetButton.disabled = value ⩵⩵ 0;
```

Code capture: js/assignment5.js; line numbers refer to the submitted source.



Result: four selected stars, matching feedback and the expanded description.

Check. Mouse selection, arrow-key selection and Clear rating passed. Reset unchecks the inputs, clears the highlight and returns focus to the first radio input. User-generated content is assigned as text, not HTML.

1. Styling and Read more

Steps. (1) Link the button to the description using aria-controls. (2) Find that element by ID. (3) Switch display between block and none. (4) Update the button text and aria-expanded together. The content remains visible when JavaScript is unavailable.

js/assignment5.js

```
37  document.querySelectorAll('[data-read-more]').forEach((button) ⇒ {
38    const content = document.getElementById(button.getAttribute('aria-controls'));
39
40    function setExpanded(expanded) {
41      content.style.display = expanded ? 'block' : 'none';
42      button.setAttribute('aria-expanded', String(expanded));
43      button.textContent = expanded ? 'Read less' : 'Read more';
44    }
45
46    button.addEventListener('click', () ⇒ {
47      setExpanded(button.getAttribute('aria-expanded') ≠≠ 'true');
48    });
```

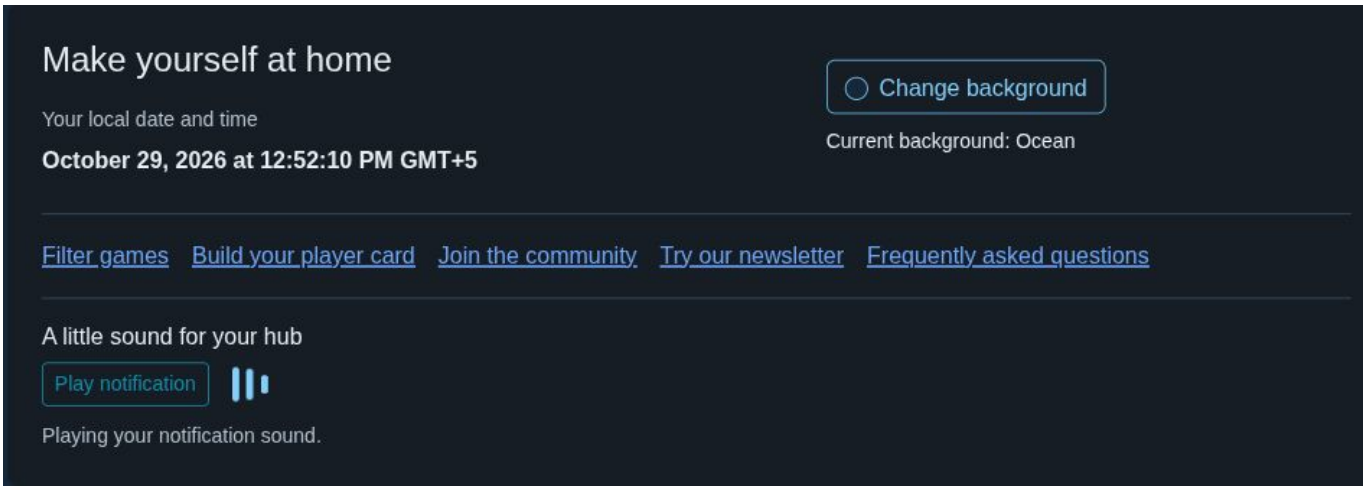
Code capture: js/assignment5.js; line numbers refer to the submitted source.

The existing background control cycles through four objects containing names and colours. It sets the body and swatch styles directly and updates the background name, satisfying the dynamic styling option.

js/site.js

```
133  const backgroundButton = document.querySelector('[data-change-background]');
134  if (backgroundButton) {
135    const backgrounds = [
136      { name: 'Midnight', color: '#101820' },
137      { name: 'Ocean', color: '#17283b' },
138      { name: 'Plum', color: '#261d38' },
139      { name: 'Forest', color: '#16342e' },
140    ];
141    let currentBackground = 0;
142    backgroundButton.addEventListener('click', () ⇒ {
143      currentBackground = (currentBackground + 1) % backgrounds.length;
144      const background = backgrounds[currentBackground];
145      document.body.style.backgroundColor = background.color;
```

Code capture: js/site.js; line numbers refer to the submitted source.



Home tools: background name and swatch update, while the sound controls remain available.

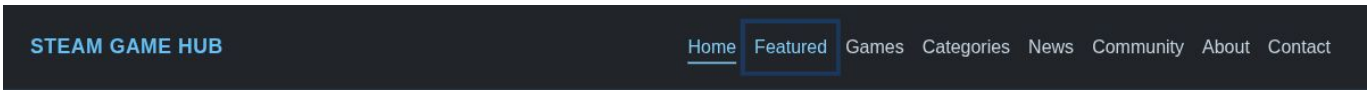
2. Click and keyboard events

Steps. (1) Attach a keydown listener to each navigation. (2) Collect only visible links. (3) Locate the focused link. (4) Use switch to choose the next index and wrap at either end. (5) Cancel arrow-key scrolling and call focus(). Enter still follows the focused link normally.

js/assignment5.js

```
55   document.querySelectorAll('nav').forEach((navigation) ⇒ {
56     navigation.addEventListener('keydown', (event) ⇒ {
57       if (event.altKey || event.ctrlKey || event.metaKey || event.shiftKey) return;
58       const links = [...navigation.querySelectorAll('a[href]')]
59         .filter((link) ⇒ link.getClientRects().length > 0);
60       const current = links.indexOf(document.activeElement);
61       if (current < 0) return;
62       let next;
63       switch (event.key) {
64         case 'ArrowRight':
65         case 'ArrowDown': next = (current + 1) % links.length; break;
66         case 'ArrowLeft':
67         case 'ArrowUp': next = (current - 1 + links.length) % links.length; break;
68         case 'Home': next = 0; break;
69         case 'End': next = links.length - 1; break;
70         default: return;
71       }
72       event.preventDefault();
73       links[next].focus();
74     });
75   });
```

Code capture: js/assignment5.js; line numbers refer to the submitted source.



Main navigation: the Featured link receives keyboard focus.

Buttons use click listeners. Reset filters restores both selects and the search field, then calls the same catalogue update function. Reusing the function keeps the UI consistent with the selected controls.

js/assignment5.js

```
129    genre.addEventListener('change', () ⇒ updateCatalogue());
130    budget.addEventListener('change', () ⇒ updateCatalogue());
131    search.addEventListener('input', () ⇒ updateCatalogue());
132    controls.querySelector('[data-reset-filters]').addEventListener('click', () ⇒ {
133      genre.value = 'all';
134      budget.value = 'all';
135      search.value = '';
136      updateCatalogue();
137    });
```

Code capture: js/assignment5.js; line numbers refer to the submitted source.

Check. Right/Down, wrapping, End and mobile menu focus passed. Hidden collapsed menu links are excluded from focus movement. Tab and modified keyboard shortcuts retain their normal browser behaviour.

2. Callback-driven form

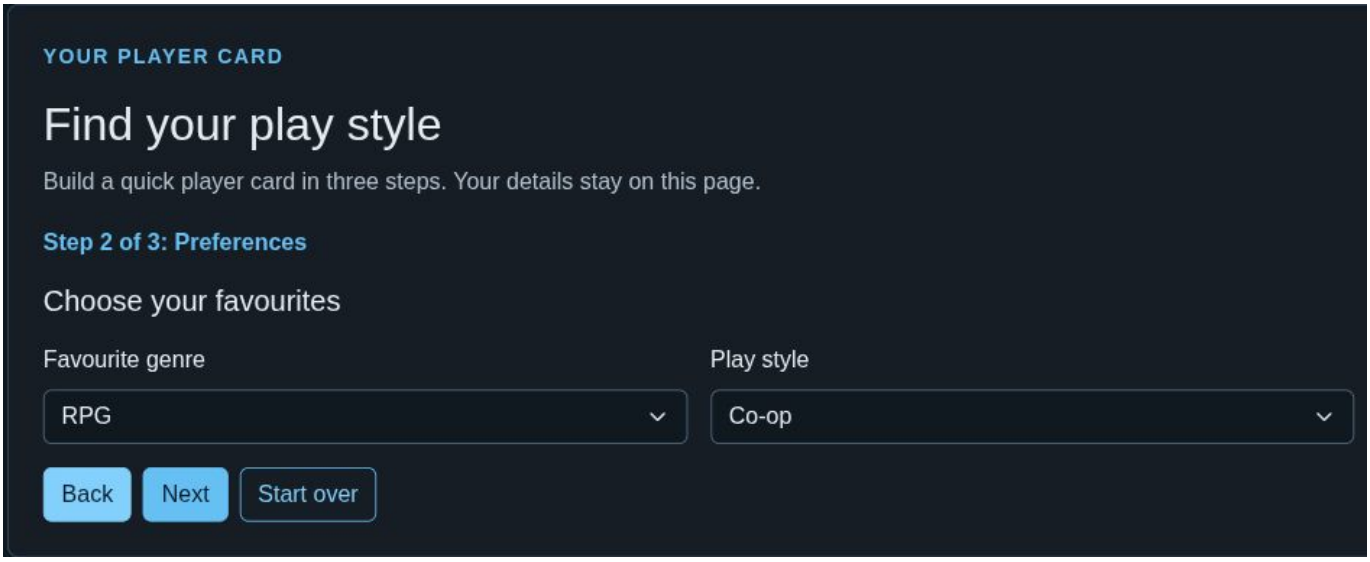
Chosen option: a three-step player profile, rather than an asynchronous contact form. Steps are Player name, Preferences and Review. No account or server endpoint is required.

Steps. (1) Store the current step in a state object. (2) Pass showStep into goTo as a callback. (3) Update the index and invoke the callback. (4) Hide inactive panels and update progress. Keeping the same input elements preserves values when moving Back and Next.

js/assignment5.js

```
155    const state = {
156      step: 0,
157      goTo(step, onStepChange) {
158        this.step = Math.max(0, Math.min(step, panels.length - 1));
159        onStepChange(this.step);
160      },
161    };
...
191    next.addEventListener('click', () ⇒ {
192      if (validateStep()) state.goTo(state.step + 1, showStep);
193    });
194    back.addEventListener('click', () ⇒ state.goTo(state.step - 1, showStep));
```

Code capture: js/assignment5.js; line numbers refer to the submitted source.



Step 2: RPG and Co-op remain selected after returning from another step.

Why this is a callback. showStep is passed as a function value, without parentheses. The parameter onStepChange receives it, then onStepChange(this.step) invokes it. A callback does not need to be asynchronous.

Check. Empty names and whitespace-only names cannot advance. Both preference fields are required. Enter on step 1 advances to step 2 without navigating away.

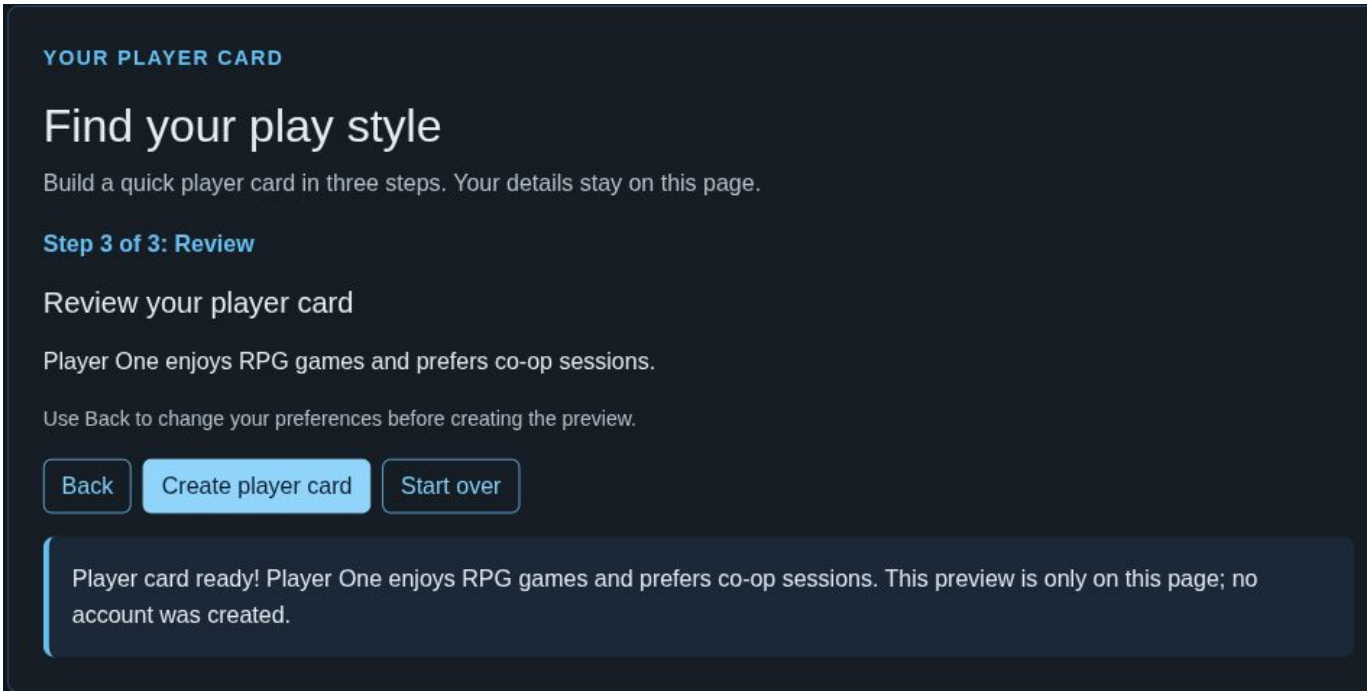
2. Validation and completion

Steps. (1) Validate only the current panel. (2) Show a field error and focus the first invalid control. (3) Build the review using textContent. (4) Prevent default submission and display a local result. (5) Start over resets fields, errors and the step index.

js/assignment5.js

```
163    function validateStep() {
164      const fields = [...panels[state.step].querySelectorAll('input, select')];
165      let firstInvalid = null;
166      for (const field of fields) {
167        const valid = field.value.trim() !== '' && field.checkValidity();
168        field.classList.toggle('is-invalid', !valid);
169        field.setAttribute('aria-invalid', String(!valid));
170        if (!valid && !firstInvalid) firstInvalid = field;
171      }
172      error.textContent = firstInvalid ? 'Please complete the highlighted field before
     continuing.' : '';
173      if (firstInvalid) firstInvalid.focus();
174      return firstInvalid === null;
```

Code capture: js/assignment5.js; line numbers refer to the submitted source.



Step 3: a player-card preview is created in place, without a page reload.

Check. Back/Next retain the name and both preferences. Start over clears all values and feedback. A name containing Player appears literally, confirming that input is not interpreted as HTML. The mobile form completes without horizontal overflow.

The original Assignment 4 registration form remains separate, preserving its required name, email, password and confirmation checks.

2-3. Switch, objects and methods

Steps. (1) Read each existing game card once. (2) Create an object containing its title, genre, numeric price and DOM element. (3) Store the objects in catalogue.games. (4) Call catalogue.findGames() whenever a filter changes. Static cards stay readable when JavaScript is disabled.

js/assignment5.js

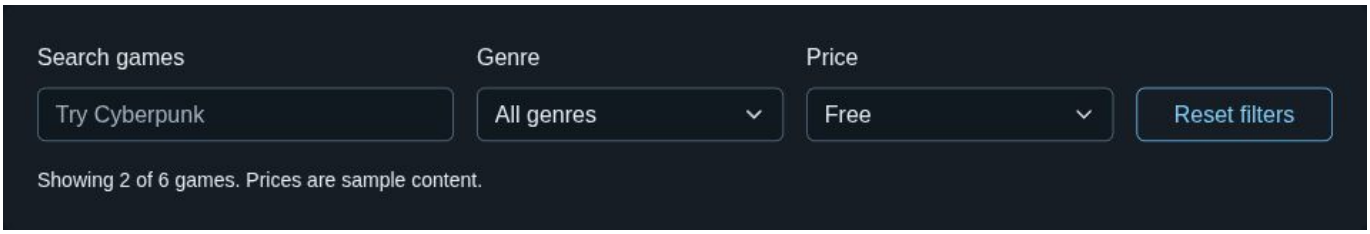
```
88     function matchesBudget(price, selection) {
89       switch (selection) {
90         case 'free': return price === 0;
91         case 'under30': return price > 0 && price < 30;
92         case 'premium': return price >= 30;
93         default: return true;
94       }
95     }
```

Code capture: js/assignment5.js; line numbers refer to the submitted source.

js/assignment5.js

```
98     const catalogue = {
99       games: [...grid.querySelectorAll('.game-card')].map((card) => ({
100        title: card.querySelector('h3').textContent.trim(),
101        genre: card.querySelector('.game-genre').textContent.trim(),
102        price: Number(card.querySelector('.price').textContent.replace(/[^\d.]/g, '')),
103        column: card.parentElement,
104      })),
105      findGames(selectedGenre, selectedBudget, query) {
106        return this.games.filter((game) =>
107          (selectedGenre === 'all' || game.genre === selectedGenre) &&
108          matchesBudget(game.price, selectedBudget) &&
109          game.title.toLowerCase().includes(query.trim().toLowerCase())
110        );
111      },
112    };
```

Code capture: js/assignment5.js; line numbers refer to the submitted source.



Price = Free displays a result count of two; filters combine with genre and search.

Higher-order function. filter receives a callback that returns true for a matching game. The callback combines genre, price and a case-insensitive title query. this.games accesses data belonging to the catalogue object.

Check. Free returns CS2 and Dota 2; paid under $30 returns GTA V and Minecraft; $30+ returns Cyberpunk 2077 and Red Dead Redemption 2. Prices are illustrative course data.

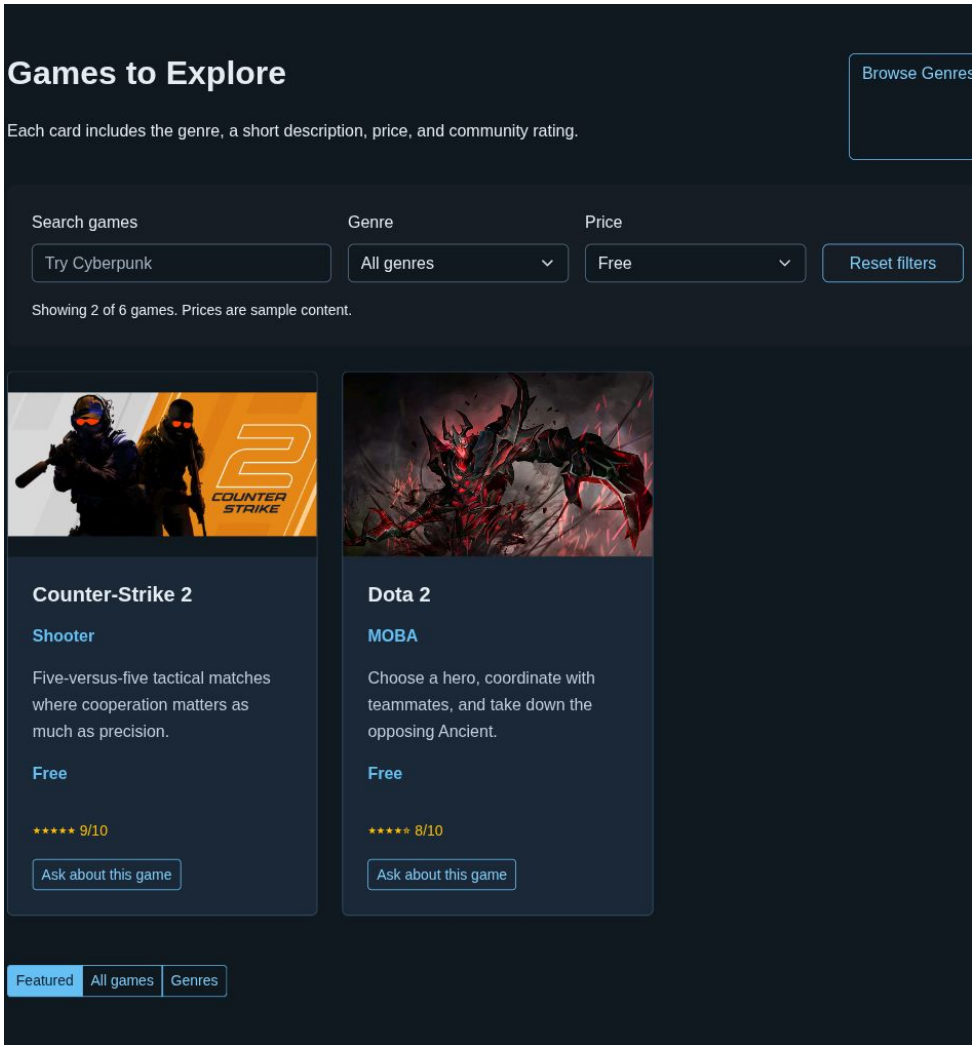
3. Arrays, loops and results

Steps. map transforms the card list into an array of game objects. findGames returns a filtered array. A for...of loop updates every card column, then visible cards receive an entrance-animation class. The result count and empty state explain the outcome.

js/assignment5.js

```
114    function updateCatalogue(animate = true) {
115      const matches = catalogue.findGames(genre.value, budget.value, search.value);
116      for (const game of catalogue.games) {
117        game.column.hidden = !matches.includes(game);
118        game.column.classList.remove('catalogue-enter');
119      }
120      // Restart the entrance animation after each filter change.
121      if (animate) {
122        void grid.offsetWidth;
123        for (const game of matches) game.column.classList.add('catalogue-enter');
124      }
```

Code capture: js/assignment5.js; line numbers refer to the submitted source.



Result of the Free price filter: two matching cards. Reset restores all six.

Check. Combined genre and budget, mixed-case search, surrounding whitespace, zero matches and Reset filters passed. The catalogue transitions are event-triggered and do not autoplay.

3. Sound and event animation

Steps. (1) Add a local 1.2-second WAV chime. (2) Call play() only inside the button handler. (3) Disable the button while playing to prevent overlap. (4) Add is-playing to animate sound bars. (5) On ended or error, restore the button and status.

js/assignment5.js

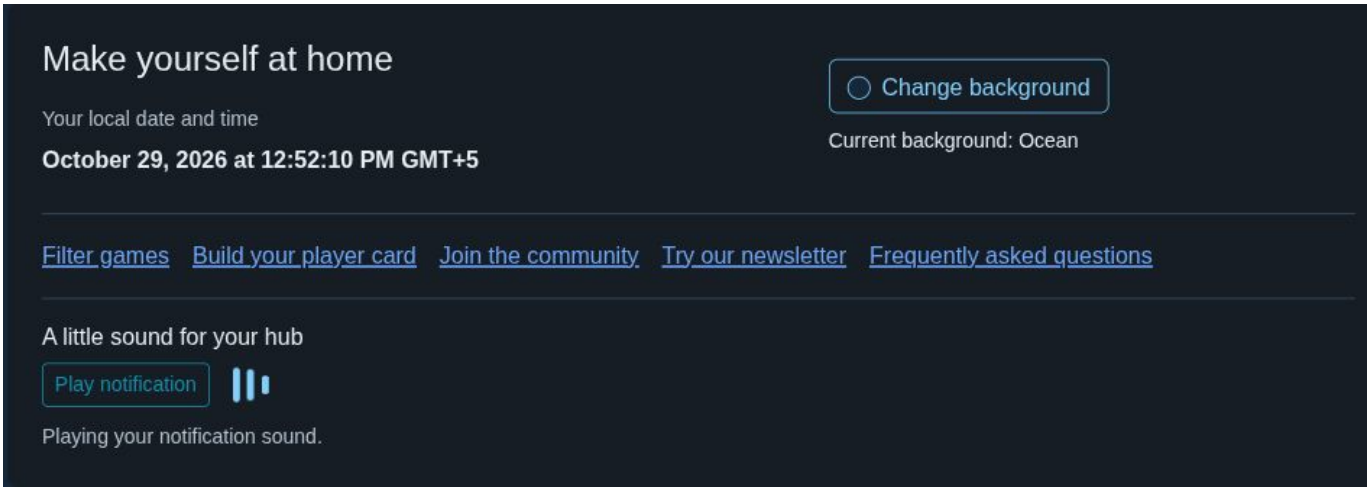
```
238    soundButton.addEventListener('click', async () => {
239      if (playing) return;
240      playing = true;
241      soundButton.disabled = true;
242      try {
243        audio.currentTime = 0;
244        await audio.play();
245        visual.classList.add('is-playing');
246        status.textContent = 'Playing your notification sound.';
247      } catch {
248        finishSound('Sound could not play. Check your browser audio settings and try
     again.');
249      }
250    });
```

Code capture: js/assignment5.js; line numbers refer to the submitted source.

css/theme.css

```
70   .catalogue-enter { animation: catalogue-reveal .35s ease both; }
71   @keyframes catalogue-reveal { from { opacity: 0; transform: translateY(12px); } to {
     opacity: 1; transform: translateY(0); } }
...
75   .sound-visual.is-playing span { animation: sound-wave .35s ease-in-out infinite
     alternate; }
76   .sound-visual.is-playing span:nth-child(2) { animation-delay: -.12s; }
77   .sound-visual.is-playing span:nth-child(3) { animation-delay: -.24s; }
78   @keyframes sound-wave { to { transform: scaleY(1); } }
```

Code capture: css/theme.css; line numbers refer to the submitted source.



Home tools include Play notification and visual feedback while audio plays.

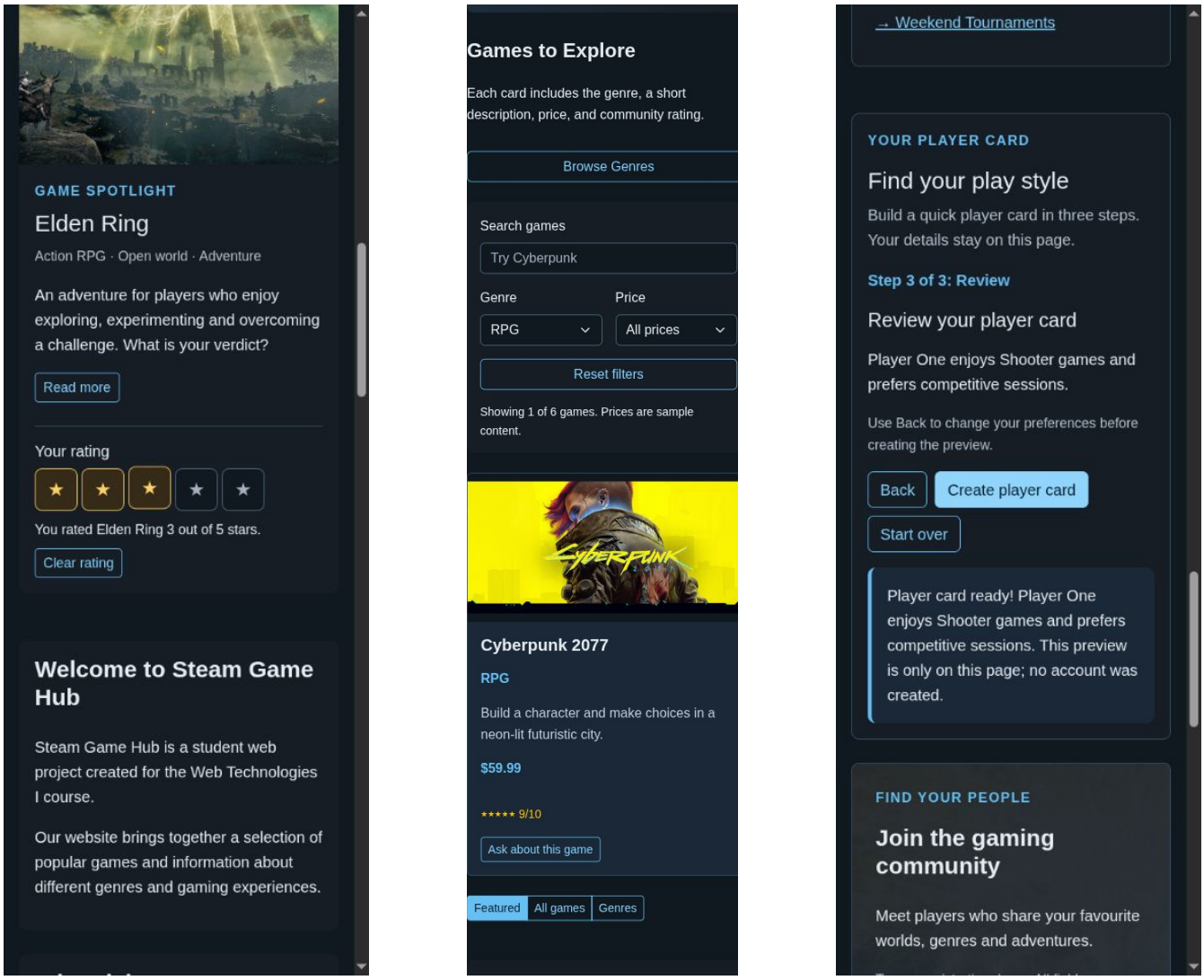
Check. The browser decoded the 1.2-second file, entered playback, disabled the button, activated animation, then restored the button after ended. CSS also disables animation and transitions under prefers-reduced-motion. Browser or system mute settings can still silence audio.

4-5. Structure, theme and UX

Separation. HTML defines labelled controls and content; css/theme.css defines appearance and motion; js/assignment5.js defines behaviour. Scripts load with defer. The existing js/site.js retains earlier features. No inline event handlers or new inline HTML style attributes are used.

Readability. Named functions such as updateRating, matchesBudget, updateCatalogue, validateStep and showStep keep repeated behaviour together. Comments identify the assignment sections and explain state, fallbacks and animation restart.

Theme and engagement. Ratings, game discovery, player preferences and a short chime fit the gaming theme. Empty states, a reset action, focus indicators, status messages and native radio controls give visible feedback.



Mobile screenshots: rating, filtered catalogue and completed player card at 375 px.

Progressive enhancement. Static game cards and descriptions remain readable without JavaScript. Interactive controls are revealed only when their handlers are ready. The new form clearly states that its result stays on the page.

Validation and deployment

71 of 71 browser checks passed on 29 October 2026. Environment: Codex in-app Chromium browser. Nine pages were inspected at 375, 768 and 1440 px. No horizontal overflow, broken loaded images, duplicate IDs or console errors were observed in those checks.

Functional coverage. Rating and details; keyboard navigation and wrapping; audio state and completion; all price branches and combined filters; zero results; multi-step validation, state preservation and reset; safe text output; mobile interactions. Regression checks also covered the existing registration, FAQ, newsletter dialog and carousel.

Evidence. report/assignment5/verification.json records the individual checks. The same folder contains webpage screenshots and source captures. The existing npm test command concerns Assignment 4; it is not the source of the 71-check Assignment 5 result.

Deployment steps. Commit the updated HTML, CSS, JavaScript, audio and evidence to the shared repository. Push main to origin. GitHub Pages builds from the root of main. Check the build result and compare the served files with the local files.

Published verification. GitHub Pages build status: built. Source commit: 5de31cbcd61b. 41 deployed files matched their local SHA-256 hashes. See deployment-verification.json for timestamps and checks.

Seven additional checks on the public site passed: rating, Read more, keyboard navigation, audio playback, price filtering, player-card completion and absence of console errors. Evidence: report/assignment5/live-browser-verification.json.

Live site: https://abilkhayrbigali-design.github.io/

ZIP contents. All nine HTML pages, css/, js/, images/, audio/, vendor/bootstrap/, README.md, this report and Assignment 5 evidence. Extract the whole folder before opening index.html. Repository metadata, dependency caches, unrelated files and earlier reports are excluded.

Submission. Each team member uploads the PDF and ZIP to LMS before the course deadline and defends the work during the practice lesson. The LMS deadline is not specified in the assignment PDF and must be checked in the course.

Reflection and defense notes

Challenge: preserving existing work. New interactions are isolated in assignment5.js and initialized only when their relevant elements exist. This allows one script to serve all pages while earlier forms and carousels continue to work.

Challenge: keeping state consistent. Hiding steps rather than replacing inputs preserves values. A single showStep callback synchronizes panels, navigation buttons, progress and review text. Reset clears values and validation feedback together.

Challenge: combined filters. Game properties are read once into objects. A single predicate combines genre, price and search, while the rendering loop handles both matching cards and the empty state. Reusing one update function prevents stale counts.

Challenge: controllable media. The audio is local and user-triggered. One audio element and a disabled trigger prevent overlapping playback; completion and error handlers restore the control. Motion has a reduced-motion CSS fallback.

What the implementation demonstrates. DOM operations connect user input to visible state. Callbacks separate step state from rendering. Objects group related values and methods. Arrays, higher-order functions and loops transform data into an updated interface.

Limits and possible next steps. This is a static academic demo. Ratings and profiles are not stored in a database, and no accounts or messages are created. A future production version would need a backend, persistent data and real catalogue information. These are outside the chosen assignment options.

Defense walkthrough. Demonstrate rating and Read more; move focus through the menu using arrows; filter Games by Free, then show an empty result; complete the player-card form using Back/Next; play the chime. Explain querySelectorAll, textContent, classList, addEventListener, switch, this, filter, for...of and the showStep callback.

Source material. Course brief: Frontend Assignment 5 JS and DOM Manipulation.pdf. Implementation: submitted project source. Image provenance: IMAGE_SOURCES.md. Bootstrap license: vendor/bootstrap/LICENSE. The notification WAV is an original synthesized three-note sound generated for this project.